

Redes Convolucionais

- I) A rede convolucional LeNET-5 ilustrada na Figura 1 foi desenvolvida por Yann LeCun e publicada em 1989. Sobre a LeNET-5, responda as perguntas abaixo.

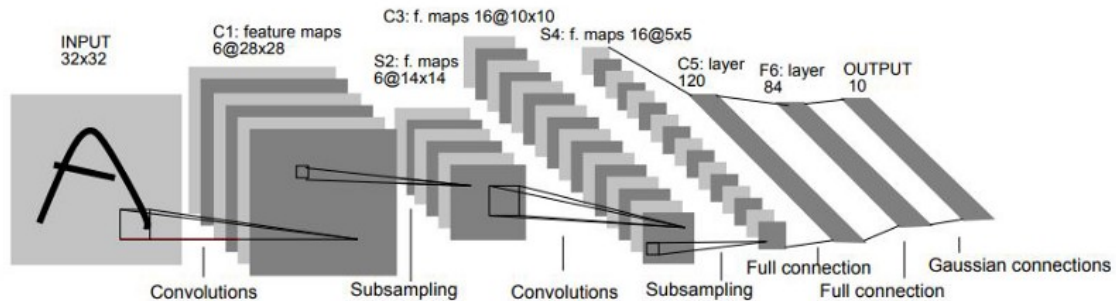


Figura 1: LeNET-5

- Qual o motivo do número 5 no nome dessa rede?
 - Quais os tamanhos das convoluções usadas?
 - Qual o tamanho do campo receptivo de uma unidade dos mapas de ativação 14×14 gerados após a camada de *average pooling* (*subsampling*)?
 - Quantos parâmetros treináveis tem essa rede?
 - A maior parte dos parâmetros treináveis está localizada nas camadas convolucionais ou nas totalmente conectadas?
- II) O que é o *stride* em uma camada convolucional?
- III) O que é o *dilation* em uma camada convolucional?
- IV) O que é uma convolução transposta?
- V) Por que dizemos que camadas de agrupamento do tipo *Max Pooling* e *Average Pooling* fazem *subsampling* dos mapas de ativação?
- VI) Por que usualmente omitimos o número de canais em uma convolução 2D?
- VII) Considerando uma convolução 2D 3×3 que conecta dois mapas de ativação com tamanho 224×224 com C_1 e C_2 canais respectivamente. Como você poderia inicializar os pesos dessa camada usando a inicialização He?
- VIII) Dada a matriz de input abaixo, e uma rede com 2 camadas convolucionais, calcule a saída da rede com as configurações a seguir. Para a primeira camada convolucional: Padding = 0, Stride = 1; Kernel = K_1 ; Ativação linear na saída. Para a segunda camada convolucional: Padding = 1; Stride = 1; Kernel = K_2 ; Ativação ReLU na saída. Considere *dilation* = 0.

$$I = \begin{bmatrix} 7 & 8 & 9 \\ 6 & 5 & 4 \\ 1 & 2 & 3 \end{bmatrix} \quad K_1 = \begin{bmatrix} 1 & 0 & -1 \\ 1 & 0 & -1 \\ 1 & 0 & -1 \end{bmatrix} \quad K_2 = \begin{bmatrix} 1 & 1 & 1 \\ 0 & 0 & 0 \\ -1 & -1 & -1 \end{bmatrix}$$

IX) Dadas as alternativas abaixo, qual das alternativas representa o Kernel mais indicado para identificação de bordas horizontais em uma imagem?

a)

$$\begin{bmatrix} -1 & -1 & -1 \\ -1 & 8 & -1 \\ -1 & -1 & -1 \end{bmatrix}$$

b)

$$\begin{bmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{bmatrix}$$

c)

$$\begin{bmatrix} 0 & -1 & 0 \\ -2 & 6 & -2 \\ 0 & -1 & 0 \end{bmatrix}$$

d)

$$\begin{bmatrix} -1 & -2 & -1 \\ 0 & 0 & 0 \\ 1 & 2 & 1 \end{bmatrix}$$

X) Dada uma imagem de entrada de 32×32 pixels, e duas camadas convolucionais, qual o campo receptivo na imagem original que é representado por um único pixel na saída final? Considere: Kernel para as duas camadas ($K_1 = K_2$) tem tamanho 5×5 ; Stride = 1; Dilation = 2.

Conexões Residuais

XI) Explique porque os dois ramos de um bloco residual não são correlacionados. Mostre que a variância da soma de duas variáveis aleatórias não correlacionadas é igual a soma das variâncias.

XII) Por que precisamos usar *batch norm* quando usamos *skip connections*?

XIII) Quantos parâmetros treináveis uma camada de *batch norm* possui?

XIV) O grafo computacional do *batch norm* é dado pela Figura 2. Nessa figura, os círculos marcam cada uma das funções que são executadas durante o processo de um forward que aplica *batch norm*. Tendo em vista esse contexto, faça o que é solicitado nas questões abaixo:

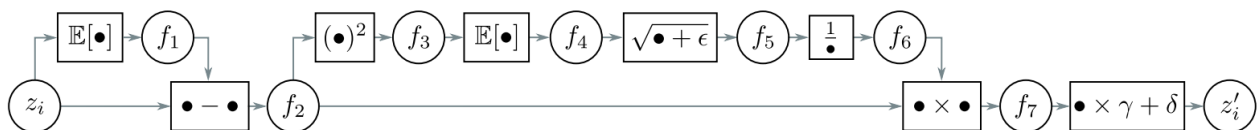


Figura 2: Grafo Computacional Batch Norm

As equações abaixo detalham o grafo computacional:

$$\begin{aligned} f_1 &= \mathbb{E}[z_i] f_{2i} = x_i - f_1 f_{3i} = f_{2i}^2 f_4 = \mathbb{E}[f_{3i}] \\ f_5 &= \sqrt{f_4 + \epsilon} f_6 = \frac{1}{f_5} f_{7i} = f_{2i} \times f_6 z'_i = f_{7i} \times \gamma + \delta \end{aligned}$$

- a) Escreva código em Python para implementar o *forward pass*.
 - b) Descubra as derivadas de cada nodo do grafo.
 - c) Crie uma expressão que computa $\frac{\partial z'_i}{\partial z_i}$
 - d) Implemente o *backward pass* em Python.
- XV) Calcule a saída do caminho $F(x)$ e a saída final de um bloco residual onde a entrada e a saída possuem as mesmas dimensões. Lembre que:

$$H(x) = F(x) + x \quad (1)$$

Onde $F(x)$ representa uma camada convolucional utilizando o Kernel K_1 . Assuma padding = 1; stride = 1; bias = 0.

$$X = \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 9 \end{bmatrix} \quad K_1 = \begin{bmatrix} 0 & 0 & 0 \\ 0 & -1 & 0 \\ 0 & 0 & 0 \end{bmatrix}$$

- XVI) Sobre a questão anterior, que aconteceria com a saída $H(x)$ se o kernel K_1 fosse treinado para ter todos os seus 9 pesos iguais a 0? O que isso significa para o treinamento de redes muito profundas?
- XVII) Em redes neurais muito profundas, os gradientes podem "desaparecer" (ficando próximos de zero), impedindo que as primeiras camadas da rede aprendam. Qual é a "chave" para resolver o problema do vanishing gradient?

Tarefas de Visão Computacional

- XVIII) Por que usamos arquiteturas baseadas em *encoder-decoder* para realizar segmentação semântica?
- XIX) Explique como as tarefas de localização e detecção de objetos são diferentes.
- XX) Explique como aproveitar uma VGG-16 pré-treinada no ImageNET para realizar classificação de cães e gatos. Sua explicação deve contemplar quais as camadas da VGG-16 você deveria manter e quais você deveria treinar novamente. Suponha que você dispõe de um conjunto de dados relativamente pequeno, com cerca de 250 instâncias para cada classe.

Gabarito

- I) a. O número 5 indica o número de camadas com parâmetros treináveis. São 2 convoluções e 3 totalmente conectadas.
- b. Na primeira camada convolucional foram convoluções 5×5 com 6 canais de saída; Na segunda foram convoluções 5×5 com 16 canais de saída.
- c. Tamanho 6×6 . Uma convolução 5×5 tem campo receptivo de tamanho 5×5 , se depois dele fizermos um *average pooling* 2×2 teremos *receptive field* de 6×6 .
- d. Na primeira camada: $5 \times 5 \times 1 \times 6 + 6 = 156$; Na segunda camada: $5 \times 5 \times 6 \times 16 + 16 = 2416$; Na terceira camada $400 \times 120 + 120 = 48120$; Na quarta camada $120 \times 80 + 80 = 10160$; Na quinta camada $84 \times 10 + 10 = 850$. Total 61702.
- e. Nas camadas totalmente conectadas.
- II) É o quanto o filtro translada entre cada uma das aplicações. Por padrão em convoluções usamos 1, o que significa que o kernel é transladado de uma posição após cada aplicação do filtro.
- III) É um parâmetro usado para aumentar o campo receptivo da convolução. Na prática tornamos o kernel esparsos completando com zeros.
- IV) É uma convolução usada para fazer *upsampling* na imagem. É uma forma de aprender a função de *upsampling*.
- V) Pois normalmente usamos essas camadas para diminuir a resolução espacial dos mapas de ativação.
- VI) Pois usualmente o número de canais da convolução é igual ao número de canais dos mapas de ativação.
- VII) A inicialização He leva em conta o número de parâmetros que contribuem para cada ativação. Como nesse caso estamos falando de $C_1 \times 3 \times 3$ pesos, podemos inicializar com uma normal com média zero e variância $\frac{2}{9C_1}$
- VIII) Resolução Input 3×3 Matrizes Iniciais

$$I = \begin{bmatrix} 7 & 8 & 9 \\ 6 & 5 & 4 \\ 1 & 2 & 3 \end{bmatrix} \quad K_1 = \begin{bmatrix} 1 & 0 & -1 \\ 1 & 0 & -1 \\ 1 & 0 & -1 \end{bmatrix} \quad K_2 = \begin{bmatrix} 1 & 1 & 1 \\ 0 & 0 & 0 \\ -1 & -1 & -1 \end{bmatrix}$$

Camada 1 (Conv1) Input 3×3 , Kernel 3×3 , $p = 0$, $s = 1 \implies$ Saída 1×1 .

$$C_1 = \sum (I \cdot K_1) + B_1 \quad (B_1 = 0)$$

$$C_1 = (7 \cdot 1 + 8 \cdot 0 + 9 \cdot -1) + (6 \cdot 1 + 5 \cdot 0 + 4 \cdot -1) + (1 \cdot 1 + 2 \cdot 0 + 3 \cdot -1)$$

$$C_1 = (-2) + (2) + (-2) = -2$$

$$C_1 = [-2]$$

Camada 2 (Conv2) Input 1×1 , Kernel 3×3 , $p = 1$, $s = 1 \implies$ Saída 1×1 . Primeiro, aplicamos padding na saída de C_1 :

$$C_{1,padded} = \begin{bmatrix} 0 & 0 & 0 \\ 0 & -2 & 0 \\ 0 & 0 & 0 \end{bmatrix}$$

Em seguida, aplicamos K_2 , $B_2 = 1$ e ReLU:

$$\text{Soma Conv} = \sum (C_{1,padded} \cdot K_2) = (-2 \cdot 0) = 0$$

$$\text{Pré ativação} = \text{Soma Conv} + B_2 = 0 + 1 = 1$$

$$\text{Output} = \text{ReLU}(1) = 1$$

Resultado Final

$$\text{Output} = [1]$$

IX) Alternativa D

X) A primeira camada de convolução com kernel 5×5 e dilatação 2, resulta em um *feature map* com campo receptivo de 9×9 sobre a imagem de entrada. A segunda camada aplica a mesma configuração de kernel sobre o *feature map* da primeira camada de convolução. Portanto, o campo receptivo final é o campo de visão da camada anterior (9×9) mais a expansão adicional do seu próprio kernel. Resultando em um campo receptivo de 17×17 .

XI) Como um dos ramos de uma conexão residual passa por uma multiplicação por um vetor de pesos seguido de uma função de ativação não-linear, as correlações lineares são perdidas.

XII) Para não causar explosão das ativações nem dos gradientes devido ao aumento da variância.

XIII) 2γ e δ

XIV) Ver aqui

XV) Cálculo de $F(x)$ (Convolução de x com K_1):

O kernel K_1 é um filtro simples que multiplica o pixel central por -1 e todos os vizinhos por 0. Ao passarmos K_1 sobre a matriz de entrada com padding, o resultado de cada operação é o valor do pixel central da entrada multiplicado por -1 .

Portanto, a matriz $F(x)$ é a negação da matriz de entrada x :

$$F(x) = \begin{bmatrix} -1 & -2 & -3 \\ -4 & -5 & -6 \\ -7 & -8 & -9 \end{bmatrix}$$

Cálculo $H(x)$: A saída final do bloco residual é a soma do resultado $F(x)$ com a entrada original x .

$$H(x) = F(x) + x$$

$$H(x) = \begin{bmatrix} -1 & -2 & -3 \\ -4 & -5 & -6 \\ -7 & -8 & -9 \end{bmatrix} + \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 9 \end{bmatrix}$$

$$H(x) = \begin{bmatrix} (-1+1) & (-2+2) & (-3+3) \\ (-4+4) & (-5+5) & (-6+6) \\ (-7+7) & (-8+8) & (-9+9) \end{bmatrix} = \begin{bmatrix} 0 & 0 & 0 \\ 0 & 0 & 0 \\ 0 & 0 & 0 \end{bmatrix}$$

Resultado Final: A saída do bloco residual é uma matriz nula:

$$H(x) = \begin{bmatrix} 0 & 0 & 0 \\ 0 & 0 & 0 \\ 0 & 0 & 0 \end{bmatrix}$$

- XVI) Se o kernel K_1 fosse treinado para ser uma matriz de zeros, e assumindo que o bias B_1 também seja zero, o resultado da convolução $F(x)$ seria uma matriz inteiramente nula. A Saída final do bloco, $H(x)$, seria calculada como:

$$H(x) = F(x) + x = \begin{bmatrix} 0 & 0 & 0 \\ 0 & 0 & 0 \\ 0 & 0 & 0 \end{bmatrix} + x = x$$

O bloco se tornaria, efetivamente, uma transformação identidade, simplesmente passando a entrada x adiante sem qualquer alteração.

- XVII) A "chave" são as conexões residuais (*skip connections*), introduzidas pelas ResNets. Elas criam um "atalho" que soma a entrada (x) diretamente à saída de um bloco de camadas ($F(x)$). Garantindo que o gradiente sempre tenha um caminho direto (o atalho $+x$) para fluir, impedindo que ele se multiplique até desaparecer.
- XVIII) Teríamos duas alternativas se não fossemos fazer com *encoder-decoder*: 1 - Usar convoluções sem fazer *downsampling*, o que ficaria inviável devido ao número de parâmetros necessários e quantidade de processamento gasto; 2 - Fazer uma rede convolucional que tem *downsampling* e rodar ela várias vezes em diferentes patches da imagem, o que seria proibitivo também em termos de custo computacional. Para realizar a segmentação da imagem em apenas uma passada, a arquitetura *encoder-decoder* possibilita toda a segmentação ser realizada em uma passada e mantém o número de parâmetros baixo.
- XIX) A localização é uma tarefa mais simples que detecção de objetos pois só estamos trabalhando com a posição do objeto de interesse na imagem, não com a determinação de sua classe. Por outro lado, na detecção de objetos tipicamente realizamos tanto a classificação e quanto a localização de objetos em uma imagem.
- XX) As camadas convolucionais da VGG-16 são responsáveis por extrair características básicas de baixo nível, como bordas e texturas, não necessitam de novo treinamento. Por outro lado, as camadas densas finais estão intimamente relacionadas ao conjunto de dados original (ImageNet). Uma abordagem usual é manter as camadas convolucionais e treinar novamente as camadas densas na extremidade da rede. Isso é um exemplo de *transfer learning* pois estamos aproveitando um aprendizado realizado em um vasto conjunto de dados para ajustar para o nosso problema específico.